

硕士论文work

论文题目：《基于视觉语言模型的病理图像诊断分析研究》

第一章 绪论

第二章 相关理论基础

第三章 基于病理基础模型知识蒸馏的视觉表征增强方法

第四章 基于强化学习的病理诊断推理能力优化研究

第五章 总结与展望

数据集:

Todo:

具体修改方案:

- 奖励规则的修改 (VQA 任务 开放式问答)
- 分类任务上的 过程级监督 (Process-Level Supervision)
 - 第一步：构建知识库 (Key-Value 映射表)
 - 第二步：奖励函数
- 数据过滤 (Hard Sample Mining) 实现

论文题目：《基于视觉语言模型的病理图像诊断分析研究》

(Research on Pathological Image Diagnosis and Analysis Based on Visual-Language Models)

第一章 绪论

- 1.1 研究背景：病理诊断需求、AI 辅助诊断现状、VLM 的机遇。
- 1.2 国内外研究现状：PFM（视觉强但无对话）、病理 VLM（能对话但视觉/推理弱）、大模型 RL（提升推理）。
- 1.3 本文研究内容：
 - 视觉增强：基于病理基础模型（PFM）蒸馏的 SFT。
 - 推理增强：基于强化学习（RL）的思维链优化。
- 1.4 论文结构。

第二章 相关理论基础

- 2.1 视觉语言模型：Qwen3-VL 架构（重点介绍其原生动态分辨率等特性）、LoRA 微调。
- 2.2 病理视觉基础模型：特征提取优势。
- 2.3 强化学习算法：PPO, GRPO, DAPO 等 的核心公式与区别。

- 2.4 评价指标：准确率、F1、BLEU-4 等。（优化本章内容和调整顺序）

第三章 基于病理基础模型知识蒸馏的视觉表征增强方法

解决“视觉表征弱”问题。1. 通过大规模病理图文描述 CPT 注入知识，2. 在大规模指令遵循数据上 通过 Visual Distillation SFT 对齐专家视觉特征。

- 3.1 引言：
 - 通用领域 VLM（Qwen3-VL）在病理细粒度特征（如细胞异型性、核分裂象）上表现不足。
 - 单纯 SFT 只能学到文本对齐，难以改变视觉编码器的表征习惯。
- 3.2 数据集构建：
 - CPT 数据：PathCap + PathGen1.6M + LLaVA（混合自然图像）。
 - SFT 数据：PathInstruct_open + PathInstruct_close (30% CoT) + detail_desc(合成的图像详细描述) + LLaVA-Instruct (混合自然图像数据) + PathVQA (Train/Val) + PathMMU数据。
(PathInstruct_open和close是Pathgen开源的指令数据)
- 3.3 方法设计：
 - 模型架构：Student (Qwen3-VL-2B-Instruct), Teacher (Conch v1.5 Frozen)。
 - 蒸馏算法：设计 **Visual Feature Distillation Loss**。

▼

Plain Text

```
1  Ltotal=Ltext(NextTokenPrediction)+λ⋅Lfeat(MSE/Cosine)Ltotal=Ltext(NextToken
    Prediction)+λ⋅Lfeat(MSE/Cosine)
```

- - 注：对齐 Qwen3 的视觉输出与 Conch 的 Image Embeddings。

- 3.4 实验与分析：
 - 基准对比：
 - Qwen3-VL (Zero-shot)。
 - Qwen3-VL + SFT (无 CPT, 无蒸馏)。
 - Qwen3-VL + CPT + SFT (无蒸馏)。
 - Ours: Qwen3-VL + CPT + SFT (with Distillation)。
 - 评估任务：
 - PathVQA & PathMMU：使用 BLEU-4, METEOR, Rouge-L 评估生成质量。
 - 目的：证明蒸馏后模型“看”得更准，描述更符合病理特征。

- 3.5 本章小结。

第四章 基于强化学习的病理诊断推理能力优化研究

定位：解决“推理逻辑弱”问题。在 Work 1 模型基础上，利用 RL 激发 CoT 能力，在分类和 MCQ 任务上实现性能跃升。

- 4.1 引言：

- SFT 模型在复杂诊断(VQA/分类)中往往直觉式回答，缺乏严谨的推理过程。
- 需要 RL 引入结果监督，强化逻辑链。

- 4.2 强化学习环境构建：

- RL 数据集：
 - 分类任务转化：Chaoyang, CCRCC, CRC-100k, CRC_MSI, BreakHis, ESCA, PanCancer-TIL。将这些分类数据构造为选择题格式 (MCQ)。
 - 复杂问答：PathInstruct_close, PathMMU(Val), PathVQA(train/val)
- 奖励函数 (Reward)：
 - Correctness Reward：答案匹配 (Regex Match) 得 1 / 0.x, 否则 0。
 - Format Reward：强制 <think>...</think><answer>...</answer> 结构。

- 4.3 强化学习策略与对比：

- 算法：GRPO, DAPO等。探讨哪种算法在病理推理任务更优。
- 基座：加载第三章训练好的 Distilled SFT 模型 作为 Actor。

- 4.4 实验与分析：

- 核心对比 (Benchmark PK)：
 - Pure PFM: Conch v1.5 + Linear Probe (代表纯视觉分类上限)。
 - Pure VLM: LLaVA-Med / Qwen3-VL (Zero-shot / SFT)。
 - Ours: CPT + Distilled SFT + RL。
 - VLM 的 Visual Encoder 提取特征 + Linear Probe.
- 评估任务：
 - 分类任务：在上述 7 个分类数据集上对比 Accuracy / F1 / AUC。（重点展示 Ours 接近或超越 PFM, 且大幅超越普通 VLM)
 - VQA 任务：PathVQA, PathMMU (Test) 的准确率。。
- 定性分析：展示 RL 训练出的 <think> 过程，证明模型学会了“先分析细胞形态，再下诊断结论”。

- 4.5 本章小结。

第五章 总结与展望

数据集:

训练数据集构建:

- **CPT 数据:** PathCap + PathGen1.6M + LLaVA (混合自然图像) 。
- **SFT 数据:** PathInstruct_open + PathInstruct_close (30% CoT + 70%MCQ) + detail_desc + LLaVA-Instruct +
PathVQA (Train/Val) + PathMMU (Val) + **分类数据集** chaoyang, CCRCC, CRC-100k, CRC_MSI, PanCancer-TIL, BreakHis
- **RL 数据**
 - **分类数据集** 同sft
 - **VQA:** PathMMU, PathVQA (同sft)
- 工作1: 评估: PathVqa, PathMMU, 工作2评估: PathVqa, PathMMU, 分类数据集
- 分类数据集: /data/ljd/Pathology_FM_LLM/expriment/classify/

Dataset	Classes
CCRCC	Blood, Cancer, Normal, Stroma
BreakHis	Benign, Malignant
Chaoyang	Normal, Serrated, Adenocarcinoma, Adenoma
CRC100K	Adipose, Background, Debris, Lymphocytes, Mucus, Muscle, Normal, Stroma, Tumor
CRC_MSI	MSIH, nonMSIH
PanCancer-TIL	TIL-Positive, TIL-Negative

训练数据集路径: /data/ljd/VLM-R1/dataset/

实验路径: /data/ljd/Pathology_FM_LLM/expriment/

Todo:

1. 蒸馏的特征 改为 vit的 patch token, 对比两种方式:
教师: CLS token -> 学生 Patch token 取平均 --- MSELoss/DINO Loss/Cosine
教师: Patch token -> 学生 Patch token 插值法 ? 一一对应后 MSELoss
2. 视觉蒸馏目前不work, 性能差异不大, 调整参数/训练策略
3. 蒸馏的教师模型可以是Conchv1.5, + UNI, Conch, UNI2 4个模型蒸馏加权

4. 对于Lingshu中的讨论方案, 进行针对性改进:
 - a. vqa任务中的奖励函数: BLEU-4 -> PubMedBERT / LLM-as-a-Judge
 - b. 分类任务的奖励函数细化: 强制模型在 **显式提及** 支撑该诊断的病理特征
 - c. 许多医学问题(简单)不需要复杂的推理。Data Curation / Filtering

具体修改方案:

奖励函数优化、过程级监督、数据筛选 对应 Lingshu 论文中的讨论

以下是针对这3点的具体实现参考:

1. 奖励规则的修改 (VQA 任务 开放式问答)

SmartPath-R1 仅使用了 BLEU-4。

引入 BERTScore (语义相似度) 或 LLM-as-a-Judge (逻辑一致性) 作为奖励。

LLM-as-a-Judge:

```
def get_llm_reward(question, model_answer, ground_truth):
    prompt = f"""
    Task: Determine if the candidate answer conveys the same medical meaning as the reference answer.
    Question: {question}
    Reference Answer: {ground_truth}
    Candidate Answer: {model_answer}

    Output 1.0 if the meaning is consistent, otherwise 0.0. Only output the number.
    """
    # 调用你的 Judge 模型 API 或本地推理
    score = judge_model.generate(prompt)
    return float(score)
```

2. 分类任务上的 过程级监督 (Process-Level Supervision)

通常的 RL (如 SmartPath-R1) 只给“最终答案对错”打分, 结果监督 (Outcome Supervision) 。

过程级监督是指给**推理过程**中的每一步或关键节点打分。在病理分类中, 因为没有现成的完美推理步骤 数据, 所以通过“关键词检查”来模拟过程监督是一种近似方法。模型不仅要答对“是癌症”, 还要在思考过程中提到“细胞核异型性”等证据才能拿满分。

具体实现步骤:

第一步: 构建知识库 (Key-Value 映射表)

需要构建一个 JSON 文件，将数据集中的 **Label类别** 映射到其**关键形态学特征**。

- **来源**：可以使用 GPT-4 批量生成，Prompt：“列出病理学中 [Label Name] 的 3-5 个核心显微镜下形态学特征关键词（英文）”。

▼ Plain Text

```
1 // pathology_knowledge_base.json
2 {
3   "Adipose Tissue": ["adipocytes", "fat cells", "large empty spaces", "chic
4     ken wire", "signet ring"],
5   "Glandular Tissue": ["acini", "ducts", "epithelial cells", "lumen", "secr
6     etory"],
7   "Lymphocytes": ["small round", "dark nuclei", "scant cytoplasm", "dense c
8     hromatin"],
9   "Carcinoma": ["atypia", "mitotic figures", "pleomorphism", "irregular nuc
10    lei"]
11 }
```

第二步：奖励函数

在 GRPO 的 Reward Function 中加入关键词匹配逻辑。

```

1  import re
2
3  # 加载知识库
4  knowledge_base = load_json("pathology_knowledge_base.json")
5
6  def process_reward_function(completions, ground_truths, **kwargs):
7      """
8      completions: 模型生成的完整文本 (包含 <think>...</think>)
9      ground_truths: 真实的分类标签列表
10     """
11     rewards = []
12
13     for completion, gt_label in zip(completions, ground_truths):
14         # 1. 提取 <think> 内容
15         think_content = extract_think_tag(completion) # 使用正则提取 <think
16         > 内部文本
17
18         # 2. 获取该标签对应的关键词列表
19         keywords = knowledge_base.get(gt_label, [])
20
21         # 3. 计算过程分: 检查关键词是否在 think_content 中出现
22         # 简单策略: 只要出现任意一个关键词就给分, 或者按出现比例给分
23         hit_count = 0
24         for kw in keywords:
25             if re.search(r'\b' + re.escape(kw) + r'\b', think_content, re.
26             IGNORECASE):
27                 hit_count += 1
28
29         # 设定过程分 R_process (例如: 命中至少一个给 0.5, 否则 0)
30         # 或者更平滑: min(hit_count / 2, 1.0) * 0.5
31         r_process = 0.5 if hit_count > 0 else 0.0
32
33         # 4. 结合结果分 (SmartPath-R1 原有的逻辑)
34         # 假设 predict_label 是从 completion 解析出来的
35         predict_label = extract_answer(completion)
36         r_label = 1.0 if predict_label == gt_label else 0.0
37
38         # 5. 总分
39         total_reward = r_label + r_process
40         rewards.append(total_reward)
41
42     return rewards

```

差异性体现: SmartPath-R1 的奖励是 **0** 或 **1**。(+0.5 或者不加 表示推理过程是否包含形态学特征描述)

3. 数据过滤 (Hard Sample Mining) 实现

如果 SFT 模型在不使用 CoT（直接回答）的情况下就能以高置信度答对，说明这个样本对于 RL 阶段学习“推理能力”是冗余的，甚至可能是有害的（模型会被迫为简单问题编造复杂的推理，即 hallucination）。

实现流程：

1. 准备阶段：

- 加载你已经训练好的 SFT 模型。
- 准备好用于 RL 的训练数据集（Prompts + Images + Labels）。

2. 推理筛选 (Inference Pass)：

- 构建一个 **Direct Answer Prompt**（强制模型不输出 `<think>`，直接输出答案）。
 - *Prompt 示例*: "Classify this image. Output the answer option directly. Do not output thinking process."
- 对训练集进行一轮批量推理（Inference）。

3. 过滤逻辑：

- 将数据集分为两部分：
 - **Easy Samples**: SFT 模型回答正确（`Prediction == GT`）。-> **剔除**（或在 RL 中大幅降权）。
 - **Hard Samples**: SFT 模型回答错误（`Prediction != GT`）。-> **保留** 用于 RL 训练。

4. 构建 RL 数据集：

- 仅使用筛选出的 **Hard Samples** 进行 GRPO 训练。

代码逻辑示意：

Python

```
1 def filter_hard_samples(sft_model, dataset):
2     hard_samples = []
3
4     for sample in dataset:
5         image = sample['image']
6         question = sample['question']
7         gt = sample['answer']
8
9         # 使用 SFT 模型进行贪婪解码 (Greedy Decoding, temperature=0)
10        # 提示词强制直接回答
11        direct_prompt = f"<image>\n{question}\nAnswer directly:"
12
13        prediction = sft_model.generate(image, direct_prompt, do_sample=False)
14
15        # 如果回答错误, 则视为“需要推理的难题”
16        if parse_answer(prediction) != gt:
17            hard_samples.append(sample)
18
19    print(f"Original size: {len(dataset)}, Filtered size: {len(hard_samples)}")
20    return hard_samples
```

创新点总结:

通过这种过滤, 避免了在简单样本上浪费了算力且引入了噪音; 而通过 Hard Sample Mining 专注于攻克模型尚未掌握的复杂样本, 从而实现了更高效的推理能力提升。